Reporte técnico de vulnerabilidad

Curso **Software Seguro** · Módulo VI — Objetivo 3: Ejecución de Laboratorios · Documento confidencial (uso autorizado)

ID del hallazgo	VULN-0006
Título	Exposición de la `SECRET_KEY` de Django en endpoint de depuración, habilitando forja de JWT y escalamiento de privilegios
Laboratorio / activo	Consulta de multas — "Escalamiento de privilegios" (entorno de práctica autorizado del curso)
URL / endpoint	`GET https://chl-<id>-consulta-multas.softwareseguro.com.ar/jsonp/?callback=…`
Severidad	**CRÍTICA — CVSS v3.1: 9.1**
Clasificación	CWE-200 / CWE-522 / CWE-269 · OWASP A05:2021 Security Misconfiguration (y A02:2021 Cryptographic Failures)
Fecha de detección	2026-11-11
Autor/a	Sofía Lourdes Toledo Criado
Estado	Resuelto en laboratorio · remediación recomendada

1. Identificación y trazabilidad

Campo	Valor
ID del hallazgo	VULN-0006
Aplicación / activo	Aplicación web "Consulta de multas" (Django + django-rest-framework-jwt, front jQuery/Bootstrap, tras Cloudflare)
URL / endpoint	`GET /jsonp/?callback=<nombre>` (fuga) → `GET /perfil/` (uso del privilegio)
Fecha de detección	2026-11-11
Reportado por	Sofía Lourdes Toledo Criado
Estado	Resuelto en laboratorio; remediación pendiente del lado del servidor
Alcance autorizado	Challenge del curso. Cuenta de prueba provista (`ignacio_123@yopmail.com`). Sin acceso a datos reales de terceros.

2. Resumen ejecutivo

Hallazgo: la aplicación expone un endpoint de depuración (`/jsonp/`) que devuelve su configuración interna, incluida la **clave secreta criptográfica** (`SECRET_KEY`) con la que se firman los tokens de sesión (JWT). Cualquier persona, **incluso sin iniciar sesión**, puede leer esa clave y a partir de ella **fabricar un token válido de cualquier usuario**, incluido el administrador.

Riesgo para el negocio: toma de control total de cuentas (account takeover). Con la clave expuesta, un atacante remoto no autenticado se convierte en administrador y puede operar funciones privilegiadas (en este laboratorio, "pagar" multas ajenas); en un sistema real equivale al compromiso de todo el mecanismo de autenticación.

Impacto principal sobre la tríada CIA: Confidencialidad e **Integridad** (altas). La disponibilidad no se vio afectada en las pruebas.

Severidad y prioridad: Crítica (CVSS v3.1 9.1). Prioridad máxima: explotable de forma remota, sin privilegios previos, sin interacción del usuario y con impacto total sobre la autenticación.

Acción recomendada: eliminar/deshabilitar de inmediato el endpoint que expone la configuración, **rotar la** `SECRET_KEY` (invalidando todos los tokens emitidos) y cargarla desde variables de entorno, nunca en código ni accesible por HTTP.

3. Detalles técnicos

3.1 Descripción

La página `/perfil/` incluye un `<script src="/jsonp/?callback=procesarDatos">` para cargar datos de configuración vía JSONP. Ese endpoint refleja el parámetro `callback` y responde con un objeto JSON que contiene parámetros sensibles del *settings* de Django, entre ellos `SECRET_KEY`, `MODO_DESARROLLO`, `TIME_ZONE`, etc. El endpoint **no exige autenticación** (responde 200 sin cookie de sesión, mientras que `/perfil/` responde 403).

La aplicación autentica mediante un JWT HS256 almacenado en la cookie `auth` (HttpOnly). La firma del JWT se realiza con la `SECRET_KEY` de Django. Al quedar esa clave expuesta, se puede **recalcular la firma HMAC-SHA256** de un token arbitrario. Se verificó que la clave filtrada reproduce **exactamente** la firma del token legítimo de la cuenta de prueba, confirmando que es la clave de firma real.

El control de rol del lado servidor se basa en el campo `email` contenido en el JWT. Forjando un token con `email = administrador_multas@yopmail.com` y firmándolo con la clave filtrada, el servidor reconoce al solicitante como administrador y revela el mensaje de éxito del desafío.

3.2 Componentes afectados

- **Aplicación / stack:** Django + `django-rest-framework-jwt` (JWT HS256; payload con `user_id`, `email`, `exp`, `orig_iat`).
- **Método y ruta de la fuga:** `GET /jsonp/?callback=<nombre>`.
- **Parámetros involucrados:** `callback` (reflejado); cookie `auth` (JWT) en el uso del privilegio.
- **Roles o permisos necesarios: ninguno** para la fuga (no autenticado). La cuenta de Ignacio solo sirve de línea de base.
- **Entorno y fecha de prueba:** instancia de laboratorio del curso, 2026-11-11.

3.3 Clasificación y causa raíz

- **CWE-200** — Exposure of Sensitive Information to an Unauthorized Actor (fuga de la configuración/clave).
- **CWE-522** — Insufficiently Protected Credentials (la clave de firma viaja por HTTP sin protección).
- **CWE-269** — Improper Privilege Management (el rol se decide por un dato forjable del token). Relacionadas: CWE-321 (uso de clave criptográfica expuesta) y CWE-798 (clave débil/predecible: `123456@pz*…`).
- **CVE:** No asignado / No aplica (debilidad de configuración propia de la aplicación).
- **Referencia OWASP:** A05:2021 Security Misconfiguration (endpoint de depuración expuesto en producción); también A02:2021 Cryptographic Failures (exposición de la clave de firma) y A07:2021 Identification and Authentication Failures (consecuencia).
- **Causa raíz probable:** endpoint de diagnóstico/JSONP dejado habilitado y sin autenticación, que serializa el *settings* completo incluyendo secretos; agravado por una `SECRET_KEY` débil y embebida.

3.4 Impacto técnico y alcance

Demostrado: un atacante remoto no autenticado obtiene la `SECRET_KEY` y forja un JWT válido de administrador, logrando escalamiento de privilegios y acceso a la función administrativa del challenge (mensaje "SOS ADMINISTRADOR" y código ganador).

Alcance potencial (no todo demostrado): en Django la `SECRET_KEY` firma además sesiones, *cookies* firmadas y tokens de restablecimiento de contraseña; su exposición implica el compromiso del **mecanismo de autenticación completo** y la suplantación de cualquier usuario. Por eso el impacto en Confidencialidad e Integridad es alto. No se probó impacto en Disponibilidad.

4. Métricas CVSS

Versión utilizada: CVSS v3.1.

Métrica	Valor / justificación
Vector CVSS completo	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`
Puntaje base	**9.1**
Severidad CVSS	Crítica

Métricas relevantes	**AV:N** (explotable por Internet); **AC:L** (una sola petición GET); **PR:N** (la fuga no requiere sesión); **UI:N** (sin interacción de víctima); **S:U** (mismo ámbito de autorización); **C:H** e **I:H** (toma de control total de cuentas); **A:N** (sin impacto de disponibilidad demostrado).
Calculadora / fuente	https://www.first.org/cvss/calculator/3.1
Prioridad interna	Crítica / inmediata — secreto de firma expuesto públicamente.

Justificación: se puntúa sobre el impacto *demostrado* (suplantación de cualquier cuenta → C:H/I:H) manteniendo `A:N` por no haberse probado afectación de disponibilidad. Nota: si se considera que el control administrativo obtenido permite destruir registros o interrumpir el servicio, correspondería `A:H`, elevando el puntaje a **9.8** con el vector `…/C:H/I:H/A:H`.

> CVSS cuantifica la severidad técnica. Aquí la prioridad de corrección es máxima porque el secreto expuesto compromete toda la autenticación y el *exploit* es trivial, remoto y silencioso.

5. Prueba de concepto (PoC) — paso a paso

Objetivo de la prueba: demostrar que, con la clave filtrada por `/jsonp/`, se forja un JWT de administrador y se accede a la función privilegiada.
Precondiciones: entorno de laboratorio autorizado; cuenta de prueba `ignacio_123@yopmail.com` (solo como línea de base; la fuga no la requiere).
Herramientas: navegador + consola de desarrollador (Web Crypto API para el HMAC-SHA256).

1. **Preparación:** se accede a la app y se identifica que `/perfil/` carga `/jsonp/?callback=procesarDatos`.
2. **Solicitud inicial (fuga):** se solicita `GET /jsonp/?callback=x` *sin cookie de sesión*. Responde 200 con la configuración, incluida `SECRET_KEY`.
3. **Modificación controlada:** se toma el payload del JWT y se cambia `email` a `administrador_multas@yopmail.com` (pista del enunciado).
4. **Ejecución (forja):** se recalcula la firma `HMAC-SHA256(header.payload, SECRET_KEY)` y se arma el JWT. Se verifica primero que la clave reproduce la firma del token legítimo de Ignacio (control de validez).
5. **Uso del privilegio:** se coloca el token forjado en la cookie `auth` y se solicita `GET /perfil/`.
6. **Verificación:** el servidor responde 200 y renderiza el banner de administrador con el código del desafío.

5.1 Evidencia

Solicitud de la fuga (sanitizada):

```
GET /jsonp/?callback=x HTTP/1.1
Host: chl-<id>-consulta-multas.softwareseguro.com.ar
(Sin cookie de sesión)
```

Respuesta relevante (secreto parcialmente ofuscado):

```
HTTP/1.1 200 OK
Content-Type: text/javascript

x({"MODO_DESARROLLO": false, "LANGUAGE_CODE": "en-us", "TIME_ZONE": "UTC",
   "USE_I18N": true, "USE_L10N": true, "USE_TZ": true,
   "SECRET_KEY": "123456@pz*+2p********"})
```

Forja y verificación del JWT (extracto del PoC en consola):

```
// 1) obtener la clave de la fuga
S = fetch('/jsonp/?callback=x').then(...).match(/"SECRET_KEY":\s*"([^"]+)"/)[1]
// 2) firmar con HMAC-SHA256 (Web Crypto)
//    control: firma recalculada del token de Ignacio == firma real  -> true
// 3) payload forjado: email = administrador_multas@yopmail.com
// 4) token = header.payload.firma   (HS256, válido)
document.cookie = 'auth=' + tokenForjado + '; path=/';   // reemplaza la sesión
location.href = '/perfil/';
```

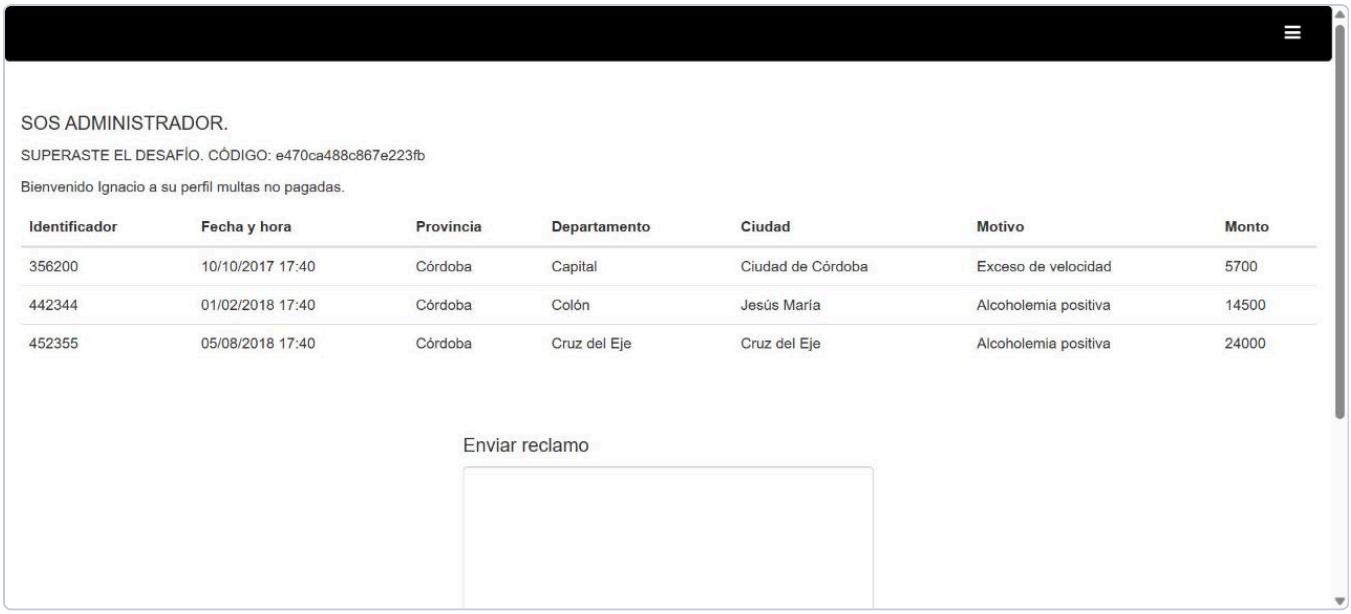


Figura 1. Tras colocar el JWT forjado en la cookie `auth`, `/perfil/` muestra "SOS ADMINISTRADOR. SUPERASTE EL DESAFÍO" con el código ganador (instancia de laboratorio). El saludo conserva el nombre de Ignacio porque el nombre se resuelve por `user_id`, pero el rol se decide por el `email` del token forjado.

- **Resultado esperado (seguro):** la configuración y la clave de firma nunca deberían ser accesibles por HTTP; un token con firma ajena debería rechazarse (403).
- **Resultado observado (vulnerable):** la clave se obtiene sin autenticación y el token forjado es aceptado como administrador.
- **Conclusión de la PoC:** queda demostrado el escalamiento de privilegios por exposición de secreto. Límite: las pruebas se realizaron sobre la instancia de laboratorio con datos ficticios.

6. Recomendaciones de remediación

6.1 Corrección inmediata

- **Eliminar o deshabilitar el endpoint** `/jsonp/` (o cualquier vista que serialice el *settings*). Nunca exponer configuración por HTTP.
- **Rotar la** `SECRET_KEY` de inmediato (invalida todos los JWT/sesiones emitidos) y generar una clave larga y aleatoria.
- **Cargar la clave desde variables de entorno / gestor de secretos**, fuera del código y del árbol web.

6.2 Medidas complementarias

- No decidir autorización a partir de campos del token que representan identidad reutilizable (`email`); usar un claim de rol validado contra el servidor o la base de datos.
- Revisar otros endpoints de diagnóstico/JSONP y desactivar el modo *debug* en producción.
- Considerar firmas asimétricas (RS256) para que el verificador no comparta el secreto de firma.
- Reducir la vigencia de los tokens (`exp`) y registrar/alertar accesos administrativos anómalos, sin exponer datos sensibles.

6.3 Criterios de aceptación

- [ ] `GET /jsonp/` ya no devuelve configuración ni secretos (404/deshabilitado).
- [ ] La `SECRET_KEY` fue rotada y se carga desde el entorno.
- [ ] Un JWT con firma inválida o forjada recibe 403.
- [ ] La autorización de administrador no depende de un campo forjable del token.
- [ ] Se ejecutaron pruebas de regresión (login legítimo sigue funcionando).

Responsable sugerido: equipo de desarrollo backend / plataforma.
Fecha objetivo: inmediata (riesgo crítico).

7. Validación posterior y seguimiento

Fecha	Responsable	Acción	Resultado
2026-11-11	Sofía Lourdes Toledo Criado	Explotación y verificación en laboratorio	Confirmado (escalamiento a administrador)
[AAAA-MM-DD]	[Equipo backend]	Retest tras remediación	Pendiente

Estado final: Abierto (remediación pendiente del lado del servidor).
Observaciones: hallazgo resuelto como challenge; el reporte documenta causa raíz y remediación para un escenario real equivalente.

Curso Software Seguro — Módulo VI, Objetivo 3. Documento confidencial de uso académico autorizado. Pruebas realizadas exclusivamente sobre el entorno de laboratorio provisto por la cátedra, con datos ficticios.